Linear Model Selection & Regularization

Subset Selection, Ridge Regression & Lasso — ISLR Chapter 6

Machine Learning with R

Dr. Endri Raço

Week 4, 2025

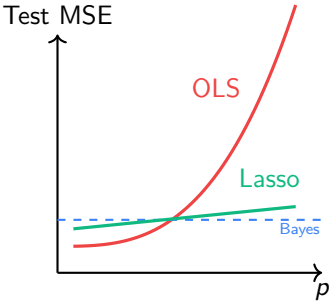
The Problem: Too Many Predictors

Weeks 1–3: OLS with all p predictors
Problem: What if p is large?

- Overfitting (low bias, high variance)
- Collinearity (unstable $\hat{\beta}$)
- $p > n$: OLS doesn't even work!
- Interpretation: which variables matter?

Three strategies:

1. Subset selection
2. Shrinkage (Ridge, Lasso)
3. Dimension reduction (PCR, PLS)



Today's Roadmap

1. Best subset selection
2. Forward/backward stepwise
3. Ridge regression
4. The Lasso
5. Elastic Net
6. Choosing λ via CV
7. PCR and PLS (preview)

Dataset: `Hitters`
$n = 263$ baseball players
$p = 19$ predictors
$Y =$ Salary (thousands \$)

Key question:
Which stats predict salary?
Can we build a **sparse**, interpretable model?

The Hitters Dataset

263 MLB players, 19 predictors:

- AtBat, Hits, HmRun, Runs, RBI
- Walks, Years, CAtBat, CHits
- CHmRun, CRuns, CRBI, CWalks
- League, Division, PutOuts
- Assists, Errors, NewLeague

59 players have missing Salary $\Rightarrow$ drop them ($n = 263$)

Challenge:

- Many predictors are correlated (career stats $\approx$ rescaled season stats)
- Which subset matters?
- OLS uses all 19 $\Rightarrow$ overfitting risk

R: `Hitters <- na.omit(Hitters)`

Best Subset Selection

Idea: Fit all 2^p possible models, pick the best

Algorithm:

1. For each $k = 0, 1, \ldots, p$:
   - Fit all $\binom{p}{k}$ models with k predictors
   - Keep the one with smallest RSS
2. Among the $p + 1$ "best" models, select using CV, C_p, BIC, or Adj. R^2

Problem: Combinatorial explosion

p	# Models
5	32
10	1,024
20	1,048,576
30	10^9

Infeasible for $p > 40$!

R: `leaps::regsubsets()`

Stepwise Selection: A Practical Alternative

Forward stepwise:

1. Start with null model ($k = 0$)
2. Add the predictor that most improves fit
3. Repeat until $k = p$
4. Select best via CV/BIC/C_p

Fits $1 + p(p+1)/2$ models

Backward stepwise:

1. Start with full model ($k = p$)
2. Remove the least useful predictor
3. Repeat until $k = 0$
4. Select best via CV/BIC/C_p

Requires $n > p$ (so OLS works)

Method	# Models fitted	Works if $p > n$?
Best subset	2^p	No
Forward	$1 + p(p+1)/2$	Yes
Backward	$1 + p(p+1)/2$	No

Choosing the Best Model: C_p, BIC, Adj. R^2

Mallow's C_p:

$$C_p = \frac{1}{n}(\text{RSS} + 2d\hat{\sigma}^2)$$

Penalizes complexity ($d = \#$ predictors)

BIC:

$$\text{BIC} = \frac{1}{n}(\text{RSS} + \log(n) \cdot d\hat{\sigma}^2)$$

Heavier penalty $\Rightarrow$ sparser models

Adjusted R^2:

$$\text{Adj. } R^2 = 1 - \frac{\text{RSS}/(n-d-1)}{\text{TSS}/(n-1)}$$

Penalizes for adding useless predictors

Best approach:

Cross-validation (Week 3)

Direct estimate of test error

No formula-based approximation needed

Subset Selection on Hitters

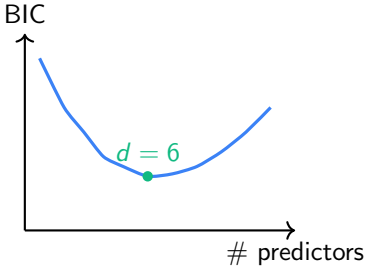
Best subset results:

d	RSS	BIC
1	36,180	−90
2	33,420	−128
3	32,110	−138
6	29,850	−148
10	28,940	−140
19	28,510	−110

BIC selects 6 predictors

C_p selects 10

Adj. R^2 selects 11



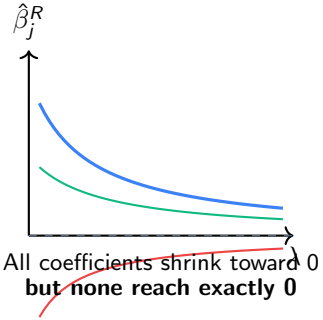
BIC penalizes more ⇒ sparser model

Ridge Regression: The Idea

OLS minimizes: $\text{RSS} = \sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
Ridge minimizes: $\text{RSS} + \lambda \sum_{j=1}^{p} \beta_j^2$

$$\hat{\beta}_\lambda^R = \arg\min_\beta \left\{ \text{RSS} + \lambda \sum_{j=1}^{p} \beta_j^2 \right\}$$

- $\lambda \geq 0$ is the **tuning parameter**
- $\lambda = 0$: identical to OLS
- $\lambda \to \infty$: all $\hat{\beta}_j \to 0$
- λ controls the bias-variance trade-off

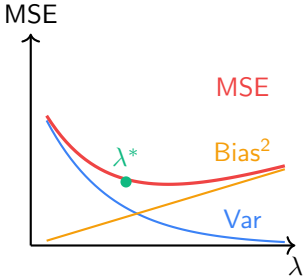


Ridge: Why Shrinking Helps

Bias-variance for ridge:

$$\text{MSE} = \text{Bias}^2 + \text{Variance}$$

- $\lambda \uparrow \Rightarrow$ Bias $\uparrow$ (moving away from OLS)
- $\lambda \uparrow \Rightarrow$ Variance $\downarrow$ (coefficients more stable)
- Optimal λ: minimizes total MSE



Especially helpful when:

- p is close to or larger than n
- Predictors are correlated
- OLS coefficients are very large/unstable

Ridge: Closed-Form Solution

$$\hat{\beta}^R = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$$

Compare with OLS:

$$\hat{\beta}^{\text{OLS}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

Ridge adds $\lambda\mathbf{I}$ to $\mathbf{X}^T\mathbf{X}$

- Makes the matrix **always invertible**
- Even when $p > n$!
- Stabilizes collinear predictors

Numerical example ($p = 2$):

$$\mathbf{X}^T\mathbf{X} = \begin{pmatrix} 10 & 9 \\ 9 & 10 \end{pmatrix}$$

Nearly singular (correlated!)

$$\mathbf{X}^T\mathbf{X} + 2\mathbf{I} = \begin{pmatrix} 12 & 9 \\ 9 & 12 \end{pmatrix}$$

Now well-conditioned!

Important: Standardize predictors before ridge! (Scale matters because of $\sum \beta_j^2$)

Why Standardize? Scale Matters

The penalty $\lambda \sum \beta_j^2$ treats all β_j equally
But β_j depends on the **scale** of X_j!

Example:

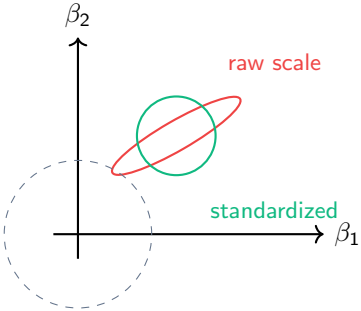
- Income in dollars: $\hat{\beta} = 0.0001$
- Income in thousands: $\hat{\beta} = 0.1$

Same predictor, different $\beta \Rightarrow$ penalty is different!

Solution: Standardize each X_j:

$$\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}$$

Now all X_j have mean 0, SD 1



Standardized: penalty circle treats all directions equally

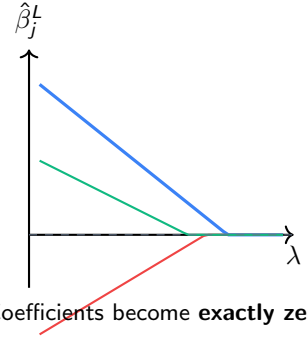
The Lasso: Sparse Models

$$\hat{\beta}^L_\lambda = \arg\min_\beta \left\{ \text{RSS} + \lambda \sum_{j=1}^{p} |\beta_j| \right\}$$

Key difference from Ridge:
$|\beta_j|$ instead of β_j^2
$\Rightarrow$ The ℓ_1 penalty can force coefficients to **exactly zero**!

Lasso = automatic variable selection
At a given λ, only a **subset** of predictors survive

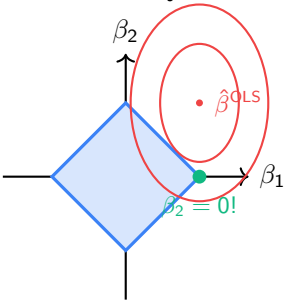
Models are **sparse** and interpretable



Coefficients become **exactly zero**!

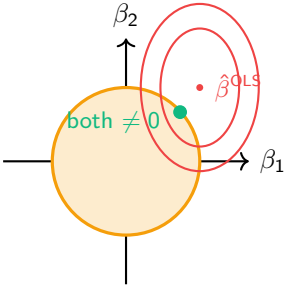
Why Does Lasso Produce Zeros? The Geometry

Lasso constraint: $|\beta_1| + |\beta_2| \leq s$
Shape: **diamond**
Corners lie on axes $\Rightarrow$ solutions land at corners $\Rightarrow$ some $\beta_j = 0$



Lasso: diamond has corners

Ridge constraint: $\beta_1^2 + \beta_2^2 \leq s$
Shape: **circle**
No corners $\Rightarrow$ solutions rarely land exactly on axis



Ridge: circle has no corners

Ridge vs. Lasso: Side-by-Side

Feature	Ridge	Lasso
Penalty	$\lambda \sum \beta_j^2$ (ℓ_2)	$\lambda \sum \lvert\beta_j\rvert$ (ℓ_1)
Constraint shape	Circle	Diamond
Coefficients $= 0$?	Never	Yes, at corners
Variable selection	No	Yes
Closed form	Yes	No (iterative)
Correlated predictors	Keeps all, shrinks evenly	Picks one, drops others
Best when	Many small effects	Few large effects

In practice, try both and use CV to decide!

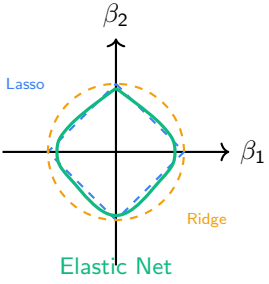
The Elastic Net: Best of Both Worlds

$$\hat{\beta}^{EN} = \arg\min_{\beta} \left\{ \text{RSS} + \lambda \left[\alpha \sum |\beta_j| + (1-\alpha) \sum \beta_j^2 \right] \right\}$$

$\alpha = 1$: pure Lasso
$\alpha = 0$: pure Ridge
$\alpha \in (0, 1)$: Elastic Net (compromise)

Advantages:

- Selects variables like Lasso
- Groups correlated variables like Ridge
- Often outperforms both individually



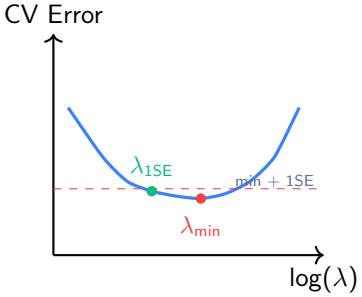
R: `glmnet(x, y, alpha=0.5)`

Choosing λ: Cross-Validation

Algorithm:

1. Create a grid of λ values
2. For each λ, compute 10-fold CV error
3. Choose λ with lowest CV error
4. Or use 1-SE rule for sparser model

`cv.glmnet()` does this automatically!



λ_{min}: lowest CV error

λ_{1SE}: most regularized within 1 SE

Worked Example: Lasso on Hitters

10-fold CV on Lasso:

$\lambda_{\text{min}} = 9.3 \Rightarrow 15$ predictors

$\lambda_{\text{1SE}} = 52.8 \Rightarrow 7$ predictors

Selected predictors (λ_{1SE}):

- AtBat, Hits, Walks
- CRuns, CRBI, CWalks
- DivisionW

12 predictors **dropped**!

Non-zero coefficients:

Predictor	$\hat{\beta}^L$
Hits	1.39
Walks	3.23
CRuns	0.73
CRBI	0.34
DivisionW	-72.8

Sparse, interpretable model!

Worked Example: Ridge vs. Lasso on Hitters

Method	CV MSE	# Predictors	Interpretation
OLS (all 19)	141,843	19	Hard
Best subset ($d = 6$)	130,220	6	Good
Ridge (λ_{min})	130,154	19	Hard
Lasso (λ_{min})	130,504	15	Medium
Lasso (λ_{1SE})	135,012	7	Easy

- Ridge and Lasso both beat OLS substantially
- Ridge and Lasso give similar CV MSE
- Lasso is more interpretable (sparse)
- Best subset is competitive but doesn't scale

Soft-Thresholding: The Lasso in a Special Case

When $\mathbf{X} = \mathbf{I}$ (orthogonal design):

Ridge:

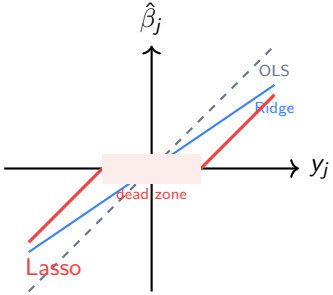
$$\hat{\beta}_j^R = \frac{y_j}{1+\lambda}$$

Proportional shrinkage

Lasso:

$$\hat{\beta}_j^L = \text{sign}(y_j)(|y_j| - \lambda/2)_+$$

Soft thresholding: small coefficients are **zeroed out**

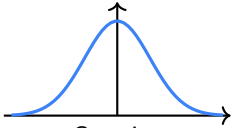


Ridge: proportional shrinkage
Lasso: threshold + shift

Bayesian Interpretation

Ridge = Gaussian prior:

$$\beta_j \sim N(0, \sigma^2/\lambda)$$

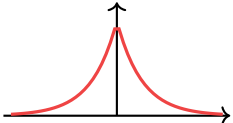


Gaussian

Smooth, bell-shaped prior
Most β_j near zero, but none exactly zero

Lasso = Laplace prior:

$$\beta_j \sim \text{Laplace}(0, 1/\lambda)$$



Laplace

Sharp peak at zero!
More probability mass at $\beta_j = 0 \Rightarrow$ promotes exact zeros

When Does Each Method Win?

Lasso wins when:

- True model is sparse
- Only a few predictors matter
- Want interpretable results
- Feature selection is the goal

few large β_j

Ridge wins when:

- Many predictors contribute
- All β_j are roughly equal
- Correlated predictor groups
- Pure prediction is the goal

many small β_j

In practice: try both (and elastic net), let CV decide

The glmnet() Function: Anatomy

```
library(glmnet)
x <- model.matrix(Salary ~ ., data)[,-1]
y <- data$Salary
# Ridge:  alpha = 0
ridge <- glmnet(x, y, alpha = 0)
# Lasso:  alpha = 1
lasso <- glmnet(x, y, alpha = 1)
# CV to choose lambda
cv.out <- cv.glmnet(x, y, alpha = 1)
best.lam <- cv.out$lambda.min
coef(cv.out, s = best.lam)
```

Key arguments:

`alpha`	0=Ridge, 1=Lasso
`lambda`	Grid of λ's
`nfolds`	# CV folds
`standardize`	TRUE (default)

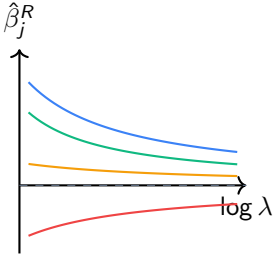
Key outputs:

`lambda.min`: optimal λ

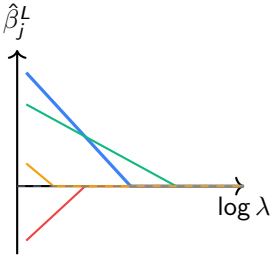
`lambda.1se`: 1-SE rule λ

`coef()`: coefficients at any λ

Coefficient Path Plots: Reading Them



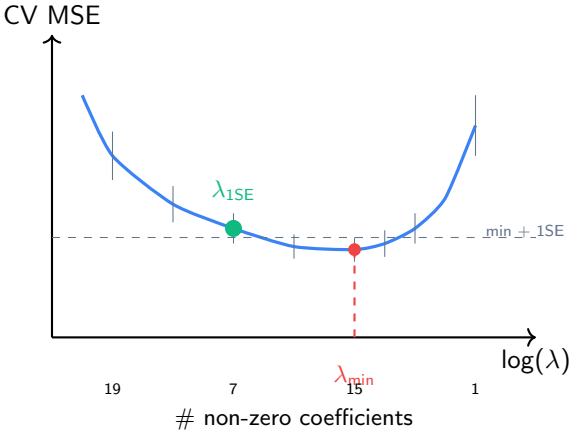
Ridge path: All shrink, **none reach 0**



Lasso path: Coefficients **hit zero** at different λ

R: `plot(glmnet_fit, xvar = "lambda")`

Cross-Validation for λ: The Full Picture



High-Dimensional Setting: $p > n$

When $p > n$:

- OLS has **no unique solution**
- $\mathbf{X}^T\mathbf{X}$ is singular
- Training RSS $= 0$ trivially (overfitting)
- $R^2 = 1$ meaningless!

Ridge and Lasso still work:

- $(\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})$ is always invertible (Ridge)
- Lasso selects at most n predictors
- Regularization is **essential** here

Example:

Genomics: $n = 100$ patients, $p = 10{,}000$ genes

Method	Possible?
OLS	No
Best subset	No
Forward step.	Yes
Ridge	Yes
Lasso	Yes

Handling Correlated Predictors

Lasso with correlated predictors:
If $X_1 \approx X_2$:
Lasso picks one and drops the other
Which one? **Essentially random!**

Ridge with correlated predictors:
Keeps both, shrinks evenly
More stable, but less sparse

Elastic Net:
Groups correlated variables together
Best of both worlds

Hitters: CRuns and CHits
$r = 0.96$ (highly correlated!)

	CRuns	CHits
OLS	1.61	-0.13
Ridge	0.82	0.71
Lasso	0.73	0

OLS: unstable (opposite signs!)
Ridge: stable, similar coefficients
Lasso: picks CRuns, drops CHits

PCR: Principal Components Regression (Preview)

Idea:

1. Compute PCA on $\mathbf{X}$ (Week 10)
2. Keep first $M < p$ components
3. Regress Y on these M components

Advantage:
Reduces p predictors to M uncorrelated components

Disadvantage:
Components are linear combinations of all $X_j \Rightarrow$ no variable selection
Choosing M via CV

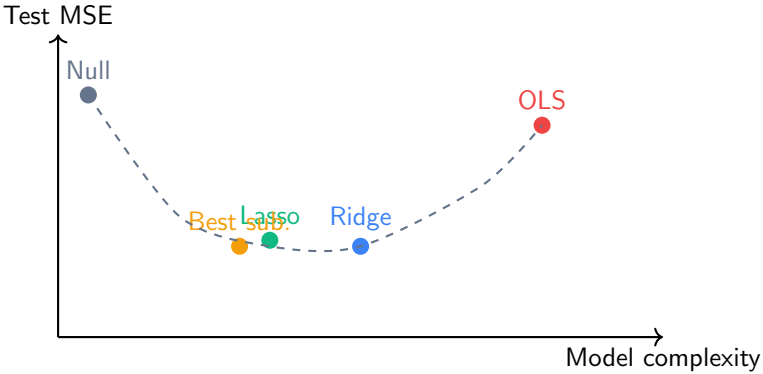
Hitters: PCR results

M	CV MSE
1	188,000
4	145,000
7	131,000
19 (OLS)	141,000

$M = 7$ components capture most of the predictive power

R: `pls::pcr(y ~ ., ncomp=7)`

Regularization: The Big Picture

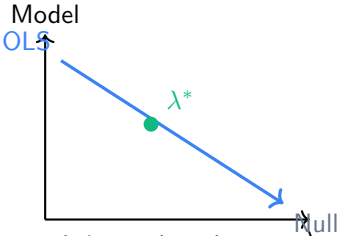


Regularization finds the **sweet spot** between the null model (too simple) and full OLS (too complex)

Think: What Does $\lambda = 0$ vs. $\lambda = \infty$ Give You?

$\lambda = 0$**:** No penalty
$\hat{\beta}^R = \hat{\beta}^{\text{OLS}}$
Full model, no shrinkage
Maximum variance, minimum bias

$\lambda = \infty$**:** Maximum penalty
$\hat{\beta}^R \to 0$ (all coefficients)
Null model: $\hat{Y} = \bar{Y}$
Minimum variance, maximum bias



λ interpolates between full OLS and the null model

Worked Example: Ridge on Hitters (Numerical)

At $\lambda = 0$ (OLS):
$\hat{\beta}_{\text{AtBat}} = -1.98$, $\hat{\beta}_{\text{Hits}} = 7.50$, $\hat{\beta}_{\text{Walks}} = 6.06$

At $\lambda = 100$:
$\hat{\beta}_{\text{AtBat}} = -0.71$, $\hat{\beta}_{\text{Hits}} = 2.73$, $\hat{\beta}_{\text{Walks}} = 2.69$

At $\lambda = 10{,}000$:
$\hat{\beta}_{\text{AtBat}} = -0.005$, $\hat{\beta}_{\text{Hits}} = 0.011$, $\hat{\beta}_{\text{Walks}} = 0.012$

λ	$\|\hat{\beta}\|_2$	# nonzero	CV MSE
0 (OLS)	228	19	141,843
100	89	19	130,154
10,000	2.3	19	178,500

Ridge: Worked Numerical Example (Mini Data)

Tiny data: $n = 4$, $p = 2$

x_1	x_2	y
1	2	5
2	1	4
3	3	8
4	2	7

(Already centered & standardized)

OLS: $\hat{\beta} = (1.4, 0.8)$

Ridge at $\lambda = 1$:

$\hat{\beta}^R = (\mathbf{X}^T\mathbf{X} + \mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$

$= (1.15, 0.65)$ (shrunken!)

Ridge at $\lambda = 10$:

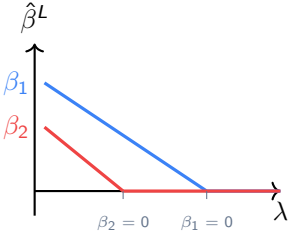
$= (0.52, 0.30)$ (more shrinkage)

Both coefficients approach zero as λ grows, but neither reaches 0

Lasso: Worked Numerical Example

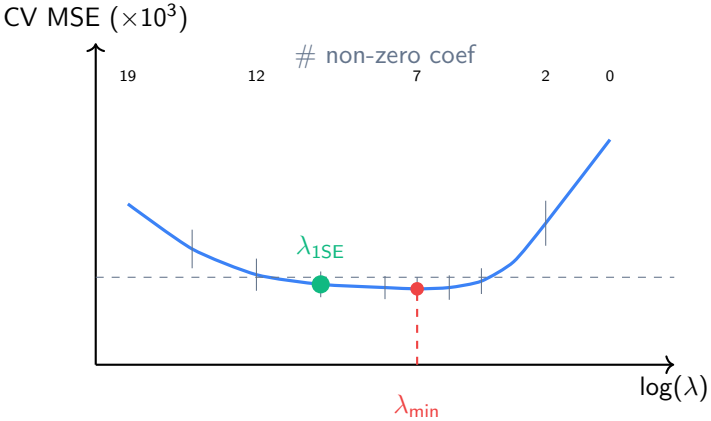
Same data, Lasso:
OLS: $\hat{\beta} = (1.4, 0.8)$
Lasso at $\lambda = 0.5$**:**
$\hat{\beta}^L = (1.15, 0.55)$
Lasso at $\lambda = 1.5$**:**
$\hat{\beta}^L = (0.65, 0)$ x_2 dropped!
Lasso at $\lambda = 3$**:**
$\hat{\beta}^L = (0, 0)$ Null model

Timeline:



x_2 drops first (smaller OLS coefficient)

Interpreting the CV Plot from `cv.glmnet`



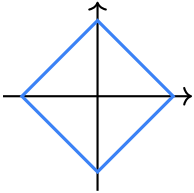
- Top axis: # predictors surviving at each λ
- Vertical bars: ± 1 standard error of the CV estimate
- Left dashed line: λ_{min} (minimum CV error)

The ℓ_1 vs. ℓ_2 Norm: Visual Intuition

ℓ_1 norm (Lasso):

$$\|\beta\|_1 = |\beta_1| + |\beta_2|$$

"Taxi-cab distance"

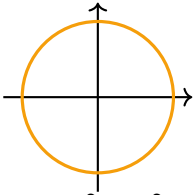


Diamond: $|\beta_1| + |\beta_2| = 2$

ℓ_2 norm (Ridge):

$$\|\beta\|_2 = \sqrt{\beta_1^2 + \beta_2^2}$$

"Euclidean distance"



Circle: $\beta_1^2 + \beta_2^2 = 4$

The diamond has **sharp corners** on the axes — that's why Lasso produces zeros!

The Constraint Formulation: A Different View

Equivalent formulations:
Ridge:

$$\min_{\beta} \text{RSS} \quad \text{subject to} \quad \sum \beta_j^2 \leq s$$

Lasso:

$$\min_{\beta} \text{RSS} \quad \text{subject to} \quad \sum |\beta_j| \leq s$$

For every λ, there exists an s that gives the same solution

Intuition:
"Minimize RSS while keeping coefficients small"

Small $s \Rightarrow$ tight constraint $\Rightarrow$ lots of shrinkage (large λ)
Large $s \Rightarrow$ loose constraint $\Rightarrow$ little shrinkage ($\lambda \approx 0$)

If s is large enough: constraint is not binding $\Rightarrow$ OLS solution

Best Subset vs. Lasso: Computational Comparison

	Best Subset	Forward	Lasso	Ridge
Computation	$O(2^p)$	$O(p^2)$	$O(p \cdot n)$	$O(p \cdot n)$
$p = 10$	1,024	55	fast	fast
$p = 20$	1M	210	fast	fast
$p = 100$	impossible	5,050	fast	fast
$p = 10{,}000$	impossible	50M	fast	fast
$p > n$?	No	Yes	Yes	Yes
Selects vars?	Yes	Yes	Yes	No

Key insight: `glmnet()` fits the **entire regularization path** (all λ's) in about the same time as a single OLS fit!

Standardization: Before and After

Before standardization:

Income: range \$10k–\$100k

Age: range 20–70

$\hat{\beta}_{\text{income}} = 0.0001$ (tiny!)

$\hat{\beta}_{\text{age}} = 50$ (huge!)

Penalty: $0.0001^2 + 50^2 = 2500$

$\Rightarrow$ Age is penalized **much more**!

After standardization:

$\tilde{X}_{\text{income}}$: mean 0, SD 1

$\tilde{X}_{\text{age}}$: mean 0, SD 1

$\hat{\beta}_{\text{income}} = 0.35$

$\hat{\beta}_{\text{age}} = 0.42$

Penalty: $0.35^2 + 0.42^2 = 0.30$

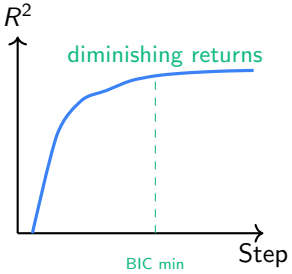
$\Rightarrow$ Both treated **equally**!

`glmnet()` standardizes by default (`standardize = TRUE`)

Coefficients are returned on the **original** scale

Forward Selection: Step-by-Step on Hitters

Step 0: Null model ($\bar{y}$)
Step 1: Try all 19 single-predictor models
Best: CRBI ($R^2 = 0.321$)
Step 2: Fix CRBI, try adding each of 18 remaining
Best: CRBI + Hits ($R^2 = 0.426$)
Step 3: Best addition: PutOuts ($R^2 = 0.458$)
⋮
Step 6: BIC minimum reached (6 predictors)



R: `regsubsets(Salary ~ ., data, method="forward")`

C_p, BIC, Adj. R^2: Hitters Results

# Predictors	RSS	C_p	BIC	Adj. R^2
1	36,180	104	−90	0.321
2	33,420	56	−128	0.411
4	30,750	18	−142	0.444
6	29,850	8	−148	0.452
8	29,380	7	−144	0.457
10	29,050	4	−140	0.460
19	28,510	20	−110	0.454

- BIC picks 6 predictors (harshest penalty)
- C_p picks 10 predictors
- Adj. R^2 picks 11 predictors (mildest penalty)
- All agree: OLS with all 19 is too many

The λ Path: What `glmnet` Actually Computes

`glmnet()` doesn't fit a single model — it fits models for a **grid of 100 λ values** simultaneously!

The trick: Warm starts

Start at $\lambda_{\max}$ (all $\hat{\beta} = 0$)
Decrease λ gradually
Use previous solution as starting point
$\Rightarrow$ Each step is very fast!

Total cost $\approx$ same as one OLS fit

λ grid:

$\lambda_{\max}$: smallest λ where all $\hat{\beta} = 0$

$$\lambda_{\max} = \frac{1}{n} \max_j |X_j^T y|$$

$\lambda_{\min} = \epsilon \cdot \lambda_{\max}$
(default $\epsilon = 0.0001$)

100 values on log scale between $\lambda_{\max}$ and $\lambda_{\min}$

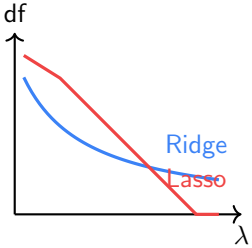
Degrees of Freedom: Ridge vs. Lasso

OLS: df $= p$ (always)
Ridge:

$$\mathrm{df}(\lambda) = \sum_{j=1}^{p} \frac{d_j^2}{d_j^2 + \lambda}$$

where d_j are singular values of $\mathbf{X}$
$\lambda = 0$: df $= p$ (OLS)
$\lambda \to \infty$: df $\to 0$ (null model)
Continuous, between 0 and p

Lasso:
df $\approx$ number of non-zero coefficients
A discrete, interpretable quantity



Elastic Net: Grouping Effect

The Lasso problem:
With correlated predictors $X_1 \approx X_2$:
Lasso picks one, drops the other (random)
$\Rightarrow$ Unstable selection

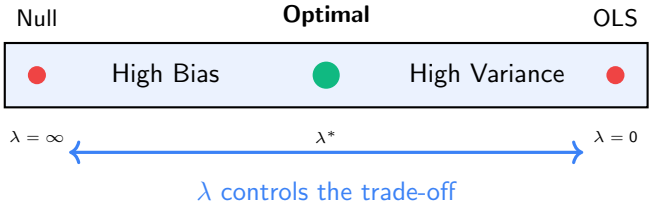
Elastic Net solves this:
The ℓ_2 component encourages correlated predictors to have similar coefficients
$\Rightarrow$ They are selected/dropped **together**
$\Rightarrow$ More stable variable selection

Example:
X_1 and X_2 have $r = 0.95$

	$\hat{\beta}_1$	$\hat{\beta}_2$
Lasso	1.2	0
EN (α=0.5)	0.7	0.6
Ridge	0.5	0.5

Elastic Net: both survive!
Coefficients are similar (grouped)

Regularization as a Bias-Variance Slider



CV finds the sweet spot where total prediction error (bias^2 + variance) is minimized

Common Mistakes with Regularization

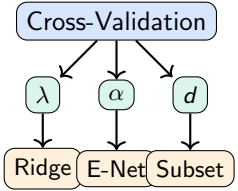
1. **Forgetting to standardize**
   - Penalty is not scale-invariant! `glmnet` does it by default
2. **Including the intercept in the penalty**
   - The intercept β_0 should **not** be penalized (it's just the mean)
   - `glmnet` handles this correctly
3. **Using training MSE to select λ**
   - Always use **cross-validation**!
4. **Reporting R^2 from regularized fit**
   - R^2 doesn't penalize for λ — use CV MSE instead
5. **Confusing α and λ in `glmnet`**
   - $\alpha =$ Ridge(0) vs Lasso(1). $\lambda =$ amount of regularization

Connecting to Week 3: CV is Essential Here

Week 3 gave us the tools:
10-fold CV estimates test error for any λ
Week 4 uses those tools to:

- Choose λ for Ridge/Lasso
- Choose α for Elastic Net
- Choose # predictors for subset selection
- Choose M for PCR

Without CV, regularization is useless!
(No way to choose λ)



CV is the universal tuning tool

Model Matrix: Converting Factors to Numerics

`glmnet()` requires a **numeric matrix**, not a data frame with factors

`model.matrix()` **converts:**

- Factors $\rightarrow$ dummy variables
- Removes intercept column (the -1)
- Keeps numeric columns as-is

```
x <- model.matrix(Salary ~ .,
    data = Hitters)[, -1]
y <- Hitters$Salary
```

Example:
League: A, N $\Rightarrow$ LeagueN (0/1)
Division: E, W $\Rightarrow$ DivisionW (0/1)
NewLeague: A, N $\Rightarrow$ NewLeagueN (0/1)

19 predictors become 19 columns (3 factors $\rightarrow$ 3 dummies + 16 numeric)

Prediction with Regularized Models

After choosing λ^*:

```
best.lam <- cv.out$lambda.min
pred <- predict(lasso.mod,
  s = best.lam, newx = x.test)
mse <- mean((y.test - pred)^2)
```

Extract final coefficients:

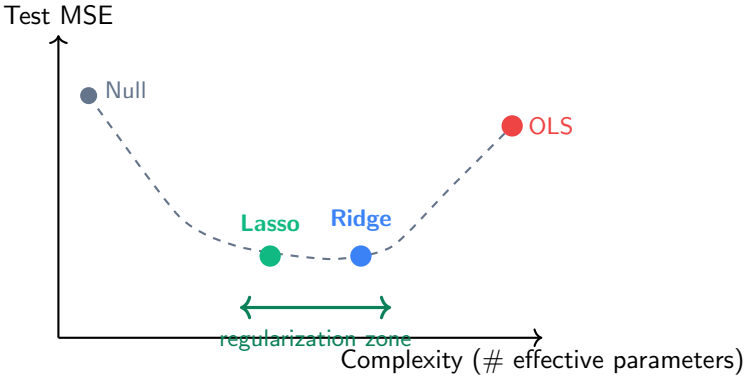
```
coef(cv.out, s = "lambda.min")
coef(cv.out, s = "lambda.1se")
```

Typical results:

Predictor	$\hat{\beta}^L$
Hits	1.39
Walks	3.23
CRuns	0.73
CRBI	0.34
DivisionW	-72.8
(12 others)	0

Sparse: only 7 of 19 survive!

Ridge vs. Lasso vs. OLS: The Final Showdown



- OLS: too far right (overfit)
- Null: too far left (underfit)
- Ridge/Lasso: in the sweet spot — both reduce test MSE!

Real-World Scenario: Gene Expression Data

Genomics: Predict disease from gene expression
$n = 100$ patients, $p = 5{,}000$ genes

OLS: Impossible ($p \gg n$)
Ridge: Test MSE $= 1.32$
Lasso: Test MSE $= 1.28$, selects 45 genes
Elastic Net: Test MSE $= 1.25$, selects 62 genes

All regularized methods work!
Lasso most interpretable (only 45 genes)

Why elastic net wins:

Gene expression data has many **groups** of correlated genes (pathways)

Lasso picks one gene per group
Elastic net keeps entire groups
$\Rightarrow$ Captures more biological signal

Regularization in Other Contexts

Regularization is everywhere:

Context	Regularization
Linear reg.	Ridge, Lasso
Logistic reg.	Penalized logistic
Neural nets	Weight decay (ℓ_2)
	Dropout
Trees	Pruning
SVM	Cost parameter C

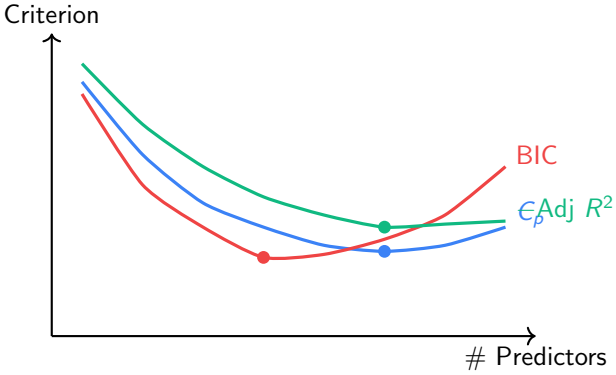
The principle is universal:

$$\min \underbrace{\text{Loss}}_{\text{fit data}} + \lambda \underbrace{\text{Penalty}}_{\text{keep simple}}$$

Every method in this course uses some form of this trade-off!

Weeks 5–10 will introduce new losses and penalties, but the core idea is the same

Comparing Selection Criteria: A Visual Guide



BIC selects fewest predictors (harshest penalty: $\log n$ vs. 2)
C_p and Adj. R^2 are similar (both use penalty $\propto d$)

Ridge: The Connection to SVD

SVD of X: $\mathbf{X} = \mathbf{U}\mathbf{D}\mathbf{V}^T$

Ridge solution:

$$\hat{\beta}^R = \sum_{j=1}^{p} \frac{d_j^2}{d_j^2 + \lambda} \cdot \frac{\mathbf{u}_j^T \mathbf{y}}{d_j} \mathbf{v}_j$$

Interpretation:

OLS projects $\mathbf{y}$ onto all p directions

Ridge **shrinks** each direction by $\frac{d_j^2}{d_j^2+\lambda}$

Small d_j (noisy directions) $\Rightarrow$ shrunk the most

Shrinkage factor:

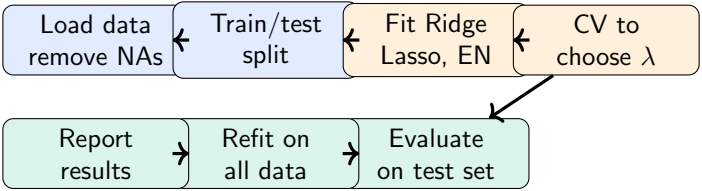
d_j	λ	Factor
10	1	0.99
1	1	0.50
0.1	1	0.01

Large singular values: barely shrunk

Small singular values: nearly zeroed out

Ridge selectively damps noise!

The Hitters Dataset: Full Analysis Pipeline



The complete workflow for any regularized regression problem

Lasso for Logistic Regression

Lasso works for **classification** too!

$$\min_{\beta} \left\{ -\ell(\beta) + \lambda \sum |\beta_j| \right\}$$

where ℓ is the logistic log-likelihood

Same idea:

Penalize large coefficients $\Rightarrow$ sparse model

R code:

```
cv.glmnet(x, y,
  family = "binomial",
  alpha = 1)
```

Default data example:

Lasso selects **balance** only (drops income and student)

	$\hat{\beta}^L$
balance	0.0054
income	0
student	0

Confirms Week 2: balance is the only useful predictor!

Elastic Net: Choosing α and λ Together

Two-dimensional tuning:

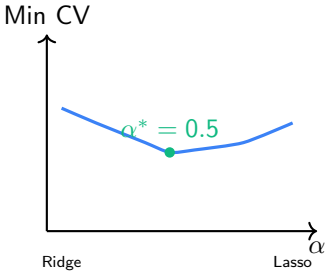
1. For each $\alpha \in \{0, 0.1, 0.2, \ldots, 1\}$:

   Run `cv.glmnet(x, y, alpha=`α`)`

   Record minimum CV error
2. Pick the (α^*, λ^*) with lowest CV error

Hitters:

α	Min CV MSE
0.0 (Ridge)	130,154
0.5 (EN)	129,800
1.0 (Lasso)	130,504



Elastic Net ($\alpha = 0.5$) wins slightly on this dataset

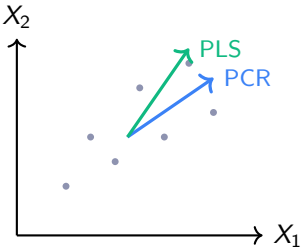
PLS: Partial Least Squares (Preview)

PCR limitation:
Components maximize variance of X
But high-variance directions of X may not predict Y!

PLS idea:
Find directions that maximize both:

- Variance of X (like PCA)
- Correlation with Y (unlike PCA)

Supervised dimension reduction!



R: `pls::plsr(y ~ ., ncomp=M)`

The Method Comparison: Complete Table

Method	Type	Selects?	$p > n$?	Tuning	Key
Best subset	Selection	Yes	No	d (via BIC)	Exhaustive
Forward	Selection	Yes	Yes	d (via CV)	Greedy
Ridge	Shrinkage	No	Yes	λ (via CV)	ℓ_2 penalty
Lasso	Shrinkage	Yes	Yes	λ (via CV)	ℓ_1 penalty
Elastic Net	Shrinkage	Yes	Yes	α, λ	Compromise
PCR	Reduction	No	Yes	M (via CV)	Unsupervised
PLS	Reduction	No	Yes	M (via CV)	Supervised

In practice: Lasso and Elastic Net are the most popular choices

Interpreting Lasso Output: A Practical Guide

After running `cv.glmnet()`**:**

1. **Plot CV curve:** Is there a clear minimum?
2. **Check λ_{min} vs $\lambda_{\mathbf{1SE}}$:** How many predictors does each select?
3. **Extract coefficients:** Which survive?
4. **Coefficient path:** When does each variable enter?
5. **Compare with OLS:** Are directions consistent?

Warning signs:

- CV curve is very flat $\Rightarrow$ regularization doesn't help much
- $\lambda_{\text{min}} \approx 0 \Rightarrow$ OLS is fine
- Very few predictors survive $\Rightarrow$ model may be too sparse
- Different runs give different selections $\Rightarrow$ use Elastic Net

Regularization Recap: The 3 Key Ideas

1. Penalize complexity

Add $\lambda \sum |\beta_j|$ or $\lambda \sum \beta_j^2$ to RSS

2. Trade bias for variance

Slight increase in bias $\Rightarrow$

3. Let CV choose the amount

λ controls the trade-off;
CV finds the sweet spot
No guesswork needed

Worked Example: Predicting College Applications

College dataset:
$n = 777$, $p = 17$
$Y =$ Applications received

Results (test set):

Method	Test MSE	# Pred
OLS	1,538k	17
Ridge	1,501k	17
Lasso	1,495k	13
EN	1,488k	14

Lasso drops:
`Books`, `Personal`, `Terminal`, `S.F.Ratio`

Key predictors:

- `Accept`: # admitted (+)
- `Top10perc`: top students (+)
- `F.Undergrad`: enrollment (+)
- `Expend`: per-student spending (+)

Intuitive and interpretable!

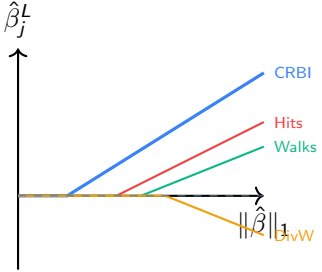
The Entire Regularization Path: A Story

Reading left to right:
λ decreases $\Rightarrow$ more predictors enter

Hitters Lasso path:

1. First in: CRBI (strongest predictor)
2. Then: Hits, Walks
3. Then: Years, CAtBat
4. Then: Division, League
5. Last: Errors, Assists (weak)

The **order** tells you importance!




```
plot(lasso.mod, xvar="norm")
```

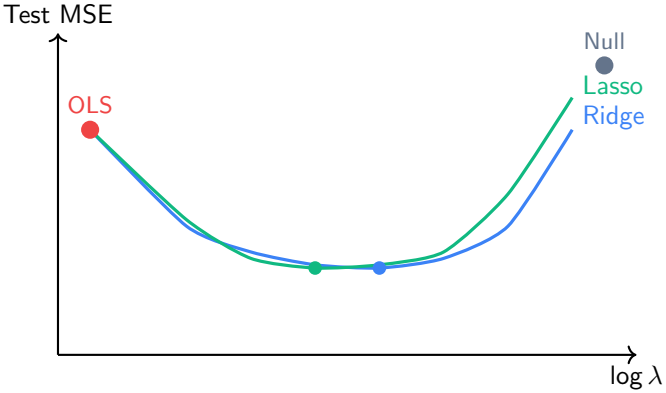
Shrinkage Effect: Before and After

Hitters: OLS vs. Ridge ($\lambda = 100$) vs. Lasso (λ_{1SE})

Predictor	OLS	Ridge	Lasso
AtBat	−1.98	−0.71	−1.87
Hits	7.50	2.73	1.39
HmRun	4.33	1.56	0
Runs	−2.38	0.42	0
RBI	−1.04	0.65	0
Walks	6.23	2.69	3.23
Years	−3.49	−1.25	0
CRBI	0.64	0.42	0.34
DivisionW	−112	−58	−73

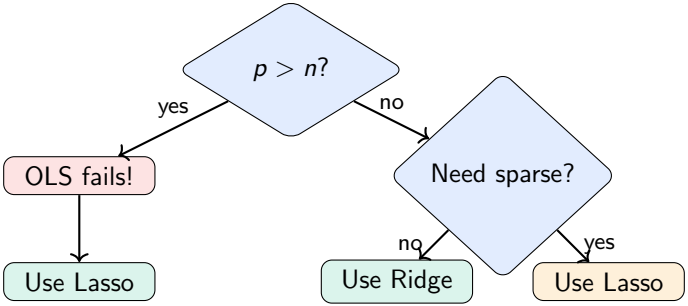
Ridge: all shrunk proportionally. Lasso: 5 of 19 zeroed out.

Test Error as a Function of λ: The Full Story



Both Ridge and Lasso produce U-shaped test error curves
The minimum occurs at different λ values but similar MSE levels

Practical Decision Tree: Which Method to Use?



Or: try both + elastic net, let CV decide

Ridge for Multicollinearity: A Concrete Example

Hitters: CRuns and CHits have $r = 0.96$

OLS:

$\hat{\beta}_{\text{CRuns}} = 1.61$, $\hat{\beta}_{\text{CHits}} = -0.13$

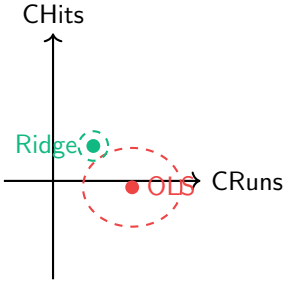
$\text{SE(CRuns)} = 0.71$, $\text{SE(CHits)} = 0.63$

Both **insignificant** despite being strong predictors!

Ridge ($\lambda = 100$):

$\hat{\beta}_{\text{CRuns}} = 0.82$, $\hat{\beta}_{\text{CHits}} = 0.71$

Now both positive and stable!



OLS: wide CI, Ridge: narrow CI

How `cv.glmnet` Handles the Folds

Inside `cv.glmnet()`:

1. Create 10 random folds
2. For each fold i:
   a. Fit `glmnet()` on 9 folds (entire λ path!)
   b. Predict on held-out fold
   c. Compute MSE at each λ
3. Average MSE across folds for each λ
4. Report λ_{min} and λ_{1SE}

Total work:
10 folds $\times$ 1 path each
$=$ 10 calls to `glmnet()`
Each call fits 100 λ values
$\Rightarrow$ 1000 models in seconds!

Reproducibility:
`set.seed()` before `cv.glmnet()` ensures same folds each time

Prediction Intervals with Regularized Models

Problem:
Ridge/Lasso don't provide standard SEs for predictions
(Shrinkage changes the distribution of $\hat{\beta}$)

Solution: Bootstrap!

1. Bootstrap the data B times
2. For each bootstrap sample:
   a. Run `cv.glmnet()` to choose λ
   b. Predict at x_0
3. Use quantiles of predictions as PI

Week 3 meets Week 4!
The bootstrap (Week 3) provides uncertainty estimates for regularized models (Week 4)

This combination is very common in practice

Caveat: computationally expensive ($B\times$ 10-fold CV)

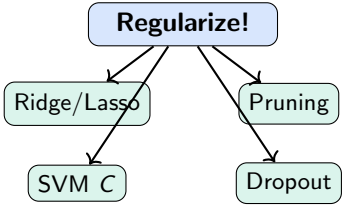
From Ridge/Lasso to Neural Networks

The regularization principle:

$$\min_{\theta} \underbrace{\text{Loss}(\theta)}_{\text{fit}} + \underbrace{\lambda \cdot \text{Pen}(\theta)}_{\text{simplify}}$$

This **same framework** extends to:

Method	Penalty
Ridge	$\sum \beta_j^2$ (weight decay)
Lasso	$\sum \lvert\beta_j\rvert$
Neural net	$\sum w_{jk}^2$ (weight decay)
Neural net	Dropout (random zeroing)
Trees	# leaves (pruning)
SVM	$\lVert\mathbf{w}\rVert^2$



Today's lesson is the template for the entire course

Summary Table: All Methods This Week

Method	Strengths	Weaknesses	R function
Best subset	Optimal if feasible	$O(2^p)$	`regsubsets()`
Forward	Fast, works if $p > n$	Greedy, may miss	`regsubsets(method="f")`
Ridge	Stable, closed form	No selection	`glmnet(alpha=0)`
Lasso	Sparse, interpretable	Unstable with r	`glmnet(alpha=1)`
Elastic Net	Groups variables	2 tuning params	`glmnet(alpha=0.5)`
PCR	Reduces dimension	Unsupervised	`pls::pcr()`

Quick Quiz: Test Yourself!

1. Ridge uses ℓ_2 penalty, Lasso uses ℓ_1. Which produces sparse models?

Quick Quiz: Test Yourself!

1. Ridge uses ℓ_2 penalty, Lasso uses ℓ_1. Which produces sparse models?

   Lasso — the ℓ_1 diamond has corners where coefficients are exactly zero

2. Why must you standardize predictors before Ridge/Lasso?

Quick Quiz: Test Yourself!

1. Ridge uses ℓ_2 penalty, Lasso uses ℓ_1. Which produces sparse models?

   Lasso — the ℓ_1 diamond has corners where coefficients are exactly zero

2. Why must you standardize predictors before Ridge/Lasso?

   The penalty $\sum \beta_j^2$ or $\sum |\beta_j|$ depends on the scale of X_j

3. $\lambda = 0$ gives what? $\lambda = \infty$ gives what?

Quick Quiz: Test Yourself!

1. Ridge uses ℓ_2 penalty, Lasso uses ℓ_1. Which produces sparse models?

   Lasso — the ℓ_1 diamond has corners where coefficients are exactly zero

2. Why must you standardize predictors before Ridge/Lasso?

   The penalty $\sum \beta_j^2$ or $\sum |\beta_j|$ depends on the scale of X_j

3. $\lambda = 0$ gives what? $\lambda = \infty$ gives what?

   $\lambda = 0$: OLS (no penalty). $\lambda = \infty$: null model (all $\beta_j = 0$)

4. Elastic net with $\alpha = 0.7$: is this closer to Ridge or Lasso?

Quick Quiz: Test Yourself!

1. Ridge uses ℓ_2 penalty, Lasso uses ℓ_1. Which produces sparse models?

   Lasso — the ℓ_1 diamond has corners where coefficients are exactly zero

2. Why must you standardize predictors before Ridge/Lasso?

   The penalty $\sum \beta_j^2$ or $\sum |\beta_j|$ depends on the scale of X_j

3. $\lambda = 0$ gives what? $\lambda = \infty$ gives what?

   $\lambda = 0$: OLS (no penalty). $\lambda = \infty$: null model (all $\beta_j = 0$)

4. Elastic net with $\alpha = 0.7$: is this closer to Ridge or Lasso?

   Closer to Lasso ($\alpha = 1$). It's 70% Lasso, 30% Ridge.

Quick Quiz: More Questions

5. With $p = 5000$ predictors and $n = 200$, can you use OLS?

Quick Quiz: More Questions

5. With $p = 5000$ predictors and $n = 200$, can you use OLS?

   No — $p > n$ so $\mathbf{X}^T\mathbf{X}$ is singular. Must use Lasso, Ridge, or forward selection.

6. Lasso selects 8 predictors at λ_{min} and 3 at λ_{1SE}. Which to use?

Quick Quiz: More Questions

5. With $p = 5000$ predictors and $n = 200$, can you use OLS?

   No — $p > n$ so $\mathbf{X}^T\mathbf{X}$ is singular. Must use Lasso, Ridge, or forward selection.

6. Lasso selects 8 predictors at λ_{min} and 3 at λ_{1SE}. Which to use?

   λ_{1SE} for parsimony (3 predictors), λ_{min} for best prediction (8 predictors)

7. Two predictors have $r = 0.99$. What happens with Lasso?

Quick Quiz: More Questions

5. With $p = 5000$ predictors and $n = 200$, can you use OLS?

   No — $p > n$ so $\mathbf{X}^T\mathbf{X}$ is singular. Must use Lasso, Ridge, or forward selection.

6. Lasso selects 8 predictors at λ_{min} and 3 at λ_{1SE}. Which to use?

   λ_{1SE} for parsimony (3 predictors), λ_{min} for best prediction (8 predictors)

7. Two predictors have $r = 0.99$. What happens with Lasso?

   Lasso arbitrarily picks one and drops the other. Use Ridge or Elastic Net instead.

8. How do you choose between Ridge and Lasso?

Quick Quiz: More Questions

5. With $p = 5000$ predictors and $n = 200$, can you use OLS?

   No — $p > n$ so $\mathbf{X}^T\mathbf{X}$ is singular. Must use Lasso, Ridge, or forward selection.

6. Lasso selects 8 predictors at λ_{min} and 3 at λ_{1SE}. Which to use?

   λ_{1SE} for parsimony (3 predictors), λ_{min} for best prediction (8 predictors)

7. Two predictors have $r = 0.99$. What happens with Lasso?

   Lasso arbitrarily picks one and drops the other. Use Ridge or Elastic Net instead.

8. How do you choose between Ridge and Lasso?

   Cross-validation! Try both (and elastic net) and pick the one with lowest CV error.

R Functions: Quick Reference

`leaps::regsubsets(y ~ ., data)`	Best subset / stepwise
`glmnet(x, y, alpha=0)`	Ridge regression
`glmnet(x, y, alpha=1)`	Lasso
`glmnet(x, y, alpha=0.5)`	Elastic net
`cv.glmnet(x, y, alpha=1)`	CV for choosing λ
`coef(cv.out, s="lambda.min")`	Coefficients at best λ
`predict(fit, s=best.lam, newx)`	Predictions
`plot(glmnet_fit, xvar="lambda")`	Coefficient path plot

Important: `glmnet()` requires a matrix `x`, not a formula!
Use `model.matrix()` to convert: `x <- model.matrix(y ~ ., data)[,-1]`

Key Formulas: Cheat Sheet

Ridge:

$$\min_{\beta} \text{RSS} + \lambda \sum_{j=1}^{p} \beta_j^2$$

$$\hat{\beta}^R = (\mathbf{X}^T\mathbf{X} + \lambda\mathbf{I})^{-1}\mathbf{X}^T\mathbf{y}$$

Lasso:

$$\min_{\beta} \text{RSS} + \lambda \sum_{j=1}^{p} |\beta_j|$$

No closed form (iterative)

Elastic Net:

$$\min_{\beta} \text{RSS} + \lambda \left[\alpha \sum |\beta_j| + (1-\alpha) \sum \beta_j^2\right]$$

Model selection:

$$C_p = \frac{1}{n}(\text{RSS} + 2d\hat{\sigma}^2)$$

$$\text{BIC} = \frac{1}{n}(\text{RSS} + \log(n) \cdot d\hat{\sigma}^2)$$

$$\text{Adj. } R^2 = 1 - \frac{\text{RSS}/(n-d-1)}{\text{TSS}/(n-1)}$$

Summary: Chapter 6

Subset Selection:

- Best subset: exhaustive but expensive
- Stepwise: practical for large p
- Use C_p, BIC, Adj. R^2, or CV to choose

Shrinkage:

- Ridge: ℓ_2, all coefficients shrunk
- Lasso: ℓ_1, sparse models
- Elastic Net: compromise
- Choose λ via CV

Key insights:

- Regularization trades bias for variance
- Essential when p is large or $> n$
- Standardize predictors first!
- CV is the universal tool for tuning

Next: Beyond Linearity — Splines & GAMs

Exercise 1: Lasso on Hitters (ISLR 6.6 Q9)

Data: `Hitters` $n = 263,\ p = 19$

1. Remove NAs: `Hitters <- na.omit(Hitters)`
2. Create `x` and `y` using `model.matrix()`
3. Fit Lasso with `cv.glmnet(x, y, alpha=1)`
4. Plot the CV curve; report λ_{min} and λ_{1SE}
5. How many non-zero coefficients at each λ?
6. Report the selected predictors and their coefficients
7. Compute test MSE using a train/test split

Exercise 2: Ridge vs. Lasso Comparison (ISLR 6.6 Q9)

1. Fit Ridge with `cv.glmnet(x, y, alpha=0)`
2. Compare Ridge CV MSE vs. Lasso CV MSE
3. Plot coefficient paths for both (`plot(fit)`)
4. Which gives a sparser model?
5. Which has lower test MSE?
6. Try Elastic Net with $\alpha = 0.5$ — how does it compare?

Exercise 3: Best Subset on Hitters (ISLR 6.6 Q9)

1. Use `regsubsets(Salary ~ ., data, nvmax=19)`
2. Plot C_p, BIC, Adj. R^2 vs. number of predictors
3. Which # predictors does each criterion select?
4. Compare with Lasso's selection
5. Implement 10-fold CV for forward stepwise

Exercise 4: College Data (ISLR 6.6 Q11)

Data: `College` Predict: `Apps` (applications received)

1. Split into train/test (70/30)
2. Fit: OLS, Ridge, Lasso, PCR
3. Compare test MSE for all four methods
4. Which method performs best?
5. How many predictors does Lasso select?

Hint: Use `pls::pcr()` for PCR with `validation = "CV"`

Solution: Exercise 1 — Lasso on Hitters

$\lambda_{\text{min}} \approx 9.3$: 15 predictors
$\lambda_{\text{1SE}} \approx 52.8$: 7 predictors

Selected at λ_{1SE}:
AtBat, Hits, Walks, CRuns, CRBI, CWalks, DivisionW

Test MSE $\approx$ 135,000

Key insight:
Lasso eliminates 12 of 19 predictors
Career stats (C-prefix) survive alongside key season stats
Division matters!

Solution: Exercise 2 — Ridge vs. Lasso

Method	CV MSE	# Nonzero	λ^*
OLS	141,843	19	—
Ridge	130,154	19	233
Lasso	130,504	15	9.3
Elastic Net	129,800	16	12.5

- All regularized methods beat OLS
- Ridge and Lasso are nearly tied
- Elastic Net slightly best (combines strengths)
- Lasso's advantage: interpretability (sparse)

Solution: Exercise 4 — College

Method	Test MSE	# Predictors
OLS	1,538,000	17
Ridge	1,501,000	17
Lasso	1,495,000	13
PCR ($M = 10$)	1,520,000	10 components

Lasso wins: lowest error AND sparse model

What's Next?

Week 3: Resampling

↓

Week 4: Regularization

↓

Week 5: Splines & GAMs

Homework: ISLR Chapter 6: Q9, 11

Lab: `Lab_Week04.R` — Ridge, Lasso, Elastic Net on Hitters + College

Next week: What if the relationship isn't linear? $\Rightarrow$ Splines!

Thank You Questions? Dr. Endri Raço — Machine Learning with R

`github.com/endri81/ISLRv2-solutions`